Samuel Julián Castillo Moñoz

Analisis de Sistemas - PLantUML

Reto1

Sala Lobby - Que es PlantUML

Codigo plantuml

```
@startuml
    Jugador -> Personaje: atacar()
@enduml
```

¿Que hace PlantUML con ese archivo de texto?

- Lo convierte en un diagrama que se puede ver y exportar, porque PlantUML interpreto las instrucciones de texto y renderiza a imgen automaticamente

Variación

```
@startuml
    Jugado -> Stehü : Seleccionar "New Round"
@enduml
```

Reto 2 - Sala Lobby - Abrir y Cerrar

- Codigo UML

```
@startuml
  title Mision: rescatar al herrero
  Guerrero -> Mision: aceptar()
  Mision --> Guerrero: recompensa = 50
@enduml
```

Tu diagrama no aparece → Plantuml marca un error en la ultima linea ¿Que revisas primero?

- Que el archivo termine con @enduml, ya que esta linea es la que indica el Fin del bloque de codigo

.

Variación

```
@startuml
  title Misión: buscar el tesoro
  Guerrero → Misión: buscar()
  Misión --> Guerrero: tesoro = 100
```

Reto 3 - Lobby - Comentarios y notas

```
@startuml
'la vida nunca baja de 0
Class Personaje
note right of Personaje
    vida entre 0 y vida Maxima
end note
@enduml
```

La linea ('la vida nunca baja de 0), no se ve en el diagrama ya que la comilla simple es un comentario

Reto 4 - Lobby - Tema y Estilo

```
@startuml
!theme cerulean
skinparam classAttributeIconSize 0
class guerrero {
  -fuerza: int
}
@enduml
```

Si cambias !theme cerulean por !theme plain. ¿Que cambia?

- Solo la apariencia del diagrama ya que el !theme solo cambia la presentación

Variación

```
@startuml
!theme dracula
class gestorInventario
@enduml
```

Reto 5

Piso 1 - Casos de uso

¿Que representa un rectangle "App Manager"?

- El limite del sistema, entre el interior de la aplicación los casos de uso y el exterior.

```
@startuml
actor Jugador
rectangle "App Manager" {
  usecase "Crear Personaje" as UC1
}
Jugador --> UC1
}
@enduml
```

Variación

```
@startuml
rectangle "Modulo de combate" {
  usecase "Usar poción" as UCP
}
Jugador --> UCP
@enduml
```

Piso 4 - Include y extend

Iniciar combate siempre obliga el equipo del Jugador. ¿Que relación usa?

- Include: Se usa cuando la ejecución de un caso de uso requiere obligatoriamente que suceda otro

Original

```
@startuml
usecase "Iniciar combate" as UC2
usecase "Validar equipo" as V
usecase "User Pasion" as P
UC2 .> V: <<include>>
P .> UC2: <<extend>>
@enduml
```

Variación

```
@startuml
usecase "comprar item" as UC
usecase "Descontar oro" as DE
UC2 .> DE: <<include>>
```

Regle

Piso 1 - El Flujo

¿Como se escribe una acción en un diagrama de actividades?

: Entrar a la arena; → las acciones se inician con (:) y finalizan con (;)

```
@startuml
start
  :Elegir personaje;
IF(Vida >0?) then (Si)
    :Entrar en la arena;
else(no)
   : Mostrar personaje caido
endif
stop
@enduml
```

Variación

```
@startuml
 start
:calcular daño;
:Aplicar multiplicador;
 stop
@enduml
```

Piso 1 - carriles

¿Para que sirven |Jugador| y |Sistema|?

- Para indicar quien realiza cada acción, se crean carriles y divide el trabajo según el actor responsable de cada tarea.

Codigo

```
@startuml
 |Jugador|
start
:Elegir Misión;
 |Sistema|
:validar nivel minimo;
 |Jugador|
:aceptar recompensa;
stop
@enduml
```

Piso 2 - Clases

¿Que significa el - antes de vida?

Atributo Privado, acceso restringido, modificable Isolo en la clase

```
@startuml
class Personaje {
-nombre:string
- vida: int
+ atacar(objeto: Personaje)
+ vivo(): bolean
}
@enduml
```

variación

```
@startuml
 class Monstruo {
 -nivel Poder: int
 + dropeo:boton()
 }
@enduml
```

Piso 3 - Relaciones

Cada personaje crea su inventario, si el personaje desaparece, el inventario tambien ¿Que flecha usas?

- Se usa *--, composición, que depende entre si, si existe una, la otra igual, si desaparece una la otra tambien

```
@startuml
abstract class Personaje
 class Guerrero
 class Inventario
 class Item
    Personaje <|-- Guerrero
    Personaje *-- Inventario
   Personaje o-- Item
@ enduml
```

variación

```
@startuml
 class Personaje
    Goerrero *-- ArmaEquipada

@enduml
```

Piso 1 - Objetos

¿Que diferencia un diagrama de objetos de uno de clases?

- Muestra instancias concretas con sus valores, el objeto ya asigna valores concretos, la clase es una plantilla

@startuml
object "aragorn: guerrero" as g {
  nombre = "Aragorn"
  vida = 120
}
object "espada: Item" as e
g -> e
@enduml

Variacion

@start uml
object "Espadafuego: Item"
as ;1 {
  daño = 50
  elemento = "Fuego"
}
@enduml

Piso 2 - Paquetes

¿Que empresa combate ..> modelo?
combate depende de modelo (lo usa), las clases del paquete modelo necesitan estas clases para funcionar

@startuml
package modelo {
  class personaje
  class item
}
package combate{
  class Arenacombate
}
combate ..> modelo

Variación

@startuml
package interFaz {
  class mpa
}
Package Logica {
  class moverjugador
}
interFaz ..> Logica

Piso 3 - Secuencia

¿Que indica la flecha punteada -->?

Un retorno o respuesta, se usa para devolver un mensaje
-> volver al actor principal

@startuml
actor jugador
Participant Arena
participant Guerrero
Jugador -> Arena : inicia.Combate()
Arena -> Guerrero : atacar(monstruo)
Guerrero --> Arena : daño = 30
@enduml

@startuml
actor Jugador
control -> calcularDaño: Daño()
calcularDaño --> control: 150

@enduml

Piso 3 - Fragmentos

La Arena repite rondas hasta que uno de los 2 caeí Que fragmento usos?

*Loop. para acciones que se repiten, es usado un bucle

@startuml

loop mientras ambos sigan vivos
Arena -> Guerrero: atacar()
alt mana >= 20
Arena -> Mago: LanzarHechizo()
else sin mana
Arena -> Mago: atagqueBasico()
end
end
@enduml

variación

@startuml
loop mientra hay rivales
Sistema -> Spawner: GenerarO
end
@enduml

Piso 3 - Estados

¿Que representa [*]?

*El inicio o Fin del ciclo de vida, entra los estados de la maquina y termina destruyendo el objeto

@startuml
[*] --> vivo
vivo --> Herido: recibirDaño
Herido --> vivo: curar
Herid --> caido: vida=0
caido --> [*]
@enduml

@startuml
[*] --> Explorando
Explorand -> EnBatalla
EnBatalla --> Muerto
Muerto [*]
@enduml

Piso 3 - Estados compuestos

¿Que ganas al poner atacando y defendiendo dentro de "En combate"?

*FinCombate sale de cualquiera de los dos sin dibujar dos flechas

```
@startuml
[*] -->EnReposo
state "En combate" as CC E
   [*] --> Atacando
   Atacando --> Defendiendo: turnoRival
   Defendiendo --> Atacando: turnoPropio
}
EnReposo --> EC: entrarAlCombate
EC --> EnReposo: FinCombate
@enduml
```

Piso 4- Componentes

¿Que muestra un diagrama de componentes?

Las piezas del software y como se conectan, muestra la arquitectura y como se conecta con los modulos, APIs, librerias

```
@startuml
component "Vista" as V
component "Servicio" as S
component "Repositorio" as R
interface "API Rest" as API
V --> API
API - S
S --> R
@enduml
```

```
@startuml
component "Vista" as V
component "Modelo" as M
component "Controlador" as C
V --> C: Solicita
C --> M: Consulta
@enduml
```

Piso 4 Despliegue

¿Que representa un nodo?

- Un equipo o entorno donde se ejecuta software, se asemeja o es un servidor Web

Codigo

```
@startuml
node "PC del estudiante"{
  artifact "rpg-manager.jar" as JAR
}
node "Servidor de Lau" {
  database "Mariadb" as DB
}
JAR --> DB: JDBC
@enduml
```

```
@startuml
node "Pc jugador" {
  component "Cliente" AS C
}
node "Servidor Cloud" {
  database "Firebase" as F
}
C --> F
@enduml
```

Piso 4 - Entidad - Relación

¿Como se lee Personaje || --o{ Item?

-Un Personaje tiene 0 o muchos items, (||) indica 1, o indica cero y { indica que el maximo es muchos

```
@startuml
entity Personaje{
 # id: INT <<PK>>
 --
 nombre: VARCHAR(50)
}
entity Item{
* id: INT <<PK>>
--
Personaje_id_: INT <<FK>>
}
Personaje ||--o{ Item
@enduml
```

y

```
@startuml
entity Jugadores{
  * id: INT <<PK>>
  --
  nombre: VARCHAR(50)
}
entity Logros{
 * id: INT <<PK>>
}
Jugadores ||--o{ Logros
@enduml
```

Piso 4 - Contenedores

¿Para que sirve !include <C4/C4_Container>?,

-Trae las macros de C4: Person, container, Rel, importa la biblioteca de Plantuml que permite usar comandos de C4

```
@startuml
!include <C4/C4_Container>
Person(jugador, "Jugador")
System_Boundary(rpg, "RPG Manager"){
    Container(web, "APP Web", "Flask")
    ContainerDb(db, "Base de datos", "MariaDB")
}
Rel(Jugador, web, "Juega en")
Rel(web, db, "Lee y escribe")
@endoml
```

Norma

Variación

```
@startuml
!include <C4/C4_Container>
Container(app, "App android", "Kotlin", "Cliente del Rpg Manage")
ContainerDb(db, "Firebase", "Nosql", "Guarda el progreso")
Rel(app, db, "Lee y escribe")
@enduml
```

Azotea Integrador Rpg Manager

¿Quieres mostrar en que orden se llama la Arena, Guerrero, Monstruo durante una ronda, ¿Que diagrama eliges T.

- Secuencial, yaque este diagrama se lee de arriba hacia abajo en orden y asi ordena los eventos.

```
@startuml
actor Jugador
Jugador -> Arena: iniciarCombate(guerrero, monstruo)
loop ronda
    Arena -> Guerrero: Atacar(monstruo)
    Arena -> Monstruo: Atacar(guerrero)
end
Arena --> Jugador: ganador
@enduml
```

Variacion

```
@startuml
Turnos -> Heroe: inicioTurno()
Heroe --> Inventario: tomarPocion()
Inventario --> Heroe: curar(50)
Turnos --> Enemigo: inicioTurno()
@enduml
```